

NUMBER SYSTEMS

Same value, different spelling

THE EXAMPLE

Building an expense tracker

Throughout this chapter we build one program: a personal expense tracker. This lecture starts with two of its numbers:

- a transaction id
- which spending categories are active this month, packed as bits

DECIMAL

The everyday case

```
int transactionId{1'000};  
std::println("Transaction id (decimal): {}", transactionId);
```

Base 10 - the numbering system we use every day. The digit separator `'` is just for readability; the compiler ignores it.

BINARY, OCTAL, HEX

The same bit pattern, three ways

```
int categoryFlags1{0b0000'1111}; // binary: 0b prefix
int categoryFlags2{017};          // octal: leading 0
int categoryFlags3{0x0F};         // hexadecimal: 0x prefix
```

All three lines store the **exact same value** - only how it's *spelled* in the source code changes.

WHY IT MATTERS

One bit per category

BIT	CATEGORY	
0	Food	
1	Transport	
2	Entertainment	
3	Other	

0b0000'1111 reads almost like a picture of which bits are set - that's the whole reason binary literals exist.

PRINTING BACK OUT

Formatting an existing value

```
std::println("transactionId as hex    : {:#x}", transactionId);  
std::println("transactionId as octal  : {:#o}", transactionId);  
std::println("transactionId as binary: {:#b}", transactionId);
```

std::println can print a number back out in whichever base is most useful for reading it - handy when hunting for which category bits are actually set.

DEBUG IT

Watch a value change base, not identity

Breakpoint after `categoryFlags1` is initialized and inspect it in your IDE's locals view - most debuggers let you switch a variable's *display* between decimal/hex/binary without changing the value stored in memory.